

Framework para el Desarrollo de Aplicaciones

Application Express

Business Intelligence

Cloud Computing

Communications

Rendimiento y Disponibilidad de Base de datos

Data Warehousing

.NET

Lenguajes de Programación Dinámicos

Embedded

Arquitectura Enterprise

Enterprise Management

Grid Computing

Identidad y Seguridad

Java

Linux

Service-Oriented Architecture

SQL & PL/SQL

Virtualización

**JMX. La guía perdida. Parte 2: Weblogic y sus MBeans Servers.**  
*Por Isaac Ruiz*  
Publicado en Enero 2017

**Alcance.**  
El presente documento detalla como conectarse utilizando código java a un servidor de MBeans contenido en un Weblogic Server 12.2.1.

Muestra, además, como se puede recuperar un valor de una propiedad de un MBEAN utilizando las APIs estándar. Adicionalmente hace una revisión por los distintos MBeans que son expuestos en WLS 12.2.1 y repasa brevemente la forma en que se asigna el *ObjectName* de cada uno de ellos.

Este documento es la 2da parte de una serie de artículos sobre JMX, puedes consultar aquí:

[Parte 1. JMX, Java Management Extensions. La guía perdida. Los fundamentos.](#)  
[Parte 3. JMX. La guía perdida. JMX, Weblogic y Multitenant.](#)  
[Parte 4. JMX. La guía perdida. Accediendo de manera remota a un proceso java.](#)

**Requerimientos.**  
El presente documento asume que se tiene cierta experiencia construyendo aplicaciones usando el lenguaje de programación java, asume también que el lector tiene experiencia como desarrollador de aplicaciones y conoce en ese nivel un sistema operativo, por ende, conoce ejecutar tareas a nivel CLI.

El documento también asume que se tiene acceso a un domino WLS 12.2.1

Si aún no tienes instalado un dominio puedes apoyarte en alguno de estos documentos:

Weblogic 12.2.1 Instalación y creación de Dominio en Windows 10.  
<https://community.oracle.com/blogs/rugi/2016/05/11/weblogic-1221-instalaci%C3%B3n-en-windows-10>  
Guía de instalación para Oracle SOA Suite 12c.  
<http://www.oracle.com/technetwork/es/articles/soa/instalacion-oracle-soa-suite-12c-2360582-esa.html>  
Instalación de Oracle SOA Suite 12c sobre Exalogic  
<http://www.oracle.com/technetwork/es/articles/soa/instalacion-soa-suite-12c-exalogic-2548201-esa.html>  
Para este ejercicio asumimos que tenemos un Dominio de desarrollo, así que solo existe un servidor, el AdminServer.

|          |                                                           |
|----------|-----------------------------------------------------------|
| URL      | <a href="http://localhost:7001">http://localhost:7001</a> |
| USER     | weblogic                                                  |
| PWD      | welcome1                                                  |
| Servidor | AdminServer                                               |

**Introducción.**  
Esta segunda parte de la serie es ligeramente más práctica. Ahora que ya conocemos un poco más sobre los componentes de la especificación podemos comenzar a utilizar esos conceptos. Para ello nos apoyaremos en un servidor Weblogic 12.2.1

WLS 12.2.1 expone varios servidores de MBeans, haremos un resumen de cuando es necesario conectarse a alguno en específico. Inspeccionar estos servidores nos obligará a comenzar a conocer los MBeans que expone cada uno de ellos.

En parte el código y las referencias expuestas en este documento son una adaptación del documento:

“Oracle® Fusion Middleware Developing Custom Management Utilities Using JMX for Oracle WebLogic Server 12.2.1”.  
<https://docs.oracle.com/middleware/1221/wls/JMXCU/title.htm>

De igual manera este documento utiliza código que toma de base el mostrado en:

“All JMX URLs”. <http://middlewaremagic.com/weblogic/?p=185>

**Weblogic y sus servidores de MBeans.**  
El objetivo de este artículo es mostrar como conectarnos remotamente a un servidor WLS 12.2.1, por ello nos hará bien recordar la siguiente imagen.

*Figura 1. Relación entre componentes y los niveles que forman la especificación JMX.*  
*Imagen tomada del documento oficial.*

En ella podemos ver que para poder acceder a un Servidor de MBean “desde afuera” de la máquina virtual que lo contiene (host1) se requiere:

Conectores.  
Adaptadores.  
¿En qué se diferencian?

**Conectores y adaptadores.**  
La especificación usa un fragmento muy corto para definir a estos dos componentes, pero, intenta ser clara en dichas definiciones

**Conector.**  
Un conector usa explícitamente todo lo definido en la especificación, involucra 2 partes para poder utilizarlo, el conector en modo server (el servidor per-se) y un cliente para comunicarse con él.

La parte cliente del conector puede utilizar distintos protocolos para comunicarse con el servidor, pero, esto realmente es transparente pues, como se mencionó anteriormente, un conector se amolda por completo a la especificación.

En términos programáticos, no importa que protocolo use el conector, la interface (jerarquía de clases) a la que tendrá acceso es la misma.

**Adaptador.**

Un adaptador por su parte, proporciona un mecanismo de comunicación con un servidor JMX en función de un protocolo específico, en otras palabras, el mecanismo es exclusivo del protocolo.

Esto es así porque, el adaptador puede exponer la información que contiene un servidor JMX utilizando distintos modelos de datos. Un ejemplo claro es SNMP.

Si vemos bien la imagen, incluso podemos reconocer que el nombre completo de un adaptador es: *Adaptador de protocolo*.

Acceder a información de un MBean server y exponer esa información en *modelos* distintos de los que hemos visto hacen que se deduzca que, el uso de adaptadores está reservado para sistemas *legacy*.

No hay que perder de vista que ya sea con conectores o adaptadores, estaremos accediendo al mismo MBean Server, solo que, si usamos conectores estaremos trabajando de manera transparente con la especificación, y si usamos adaptadores tendremos que ajustarnos al mecanismo (y las limitantes) que el protocolo utilizado nos exponga.

#### Conectores de la plataforma.

Usaremos el conector que proporciona la plataforma, por ello conoceremos como identificar un servidor y un cliente para el conector.

#### Conexión remota a un WLS.

En el documento < Developing Custom Management Utilities Using JMX >, en el apartado < Make Remote Connections to an MBean Server > se describen los pasos necesarios para realiza una conexión remota a un servidor JXM que se encuentra hospedado por un WLS 12.2.1

#### Listado de servidores.

Lo primero que debemos saber antes de iniciar a codificar es que WLS 12.2.1 expone no uno sino tres servidores JMX.

Los cuales se describen en la siguiente tabla:

| MBean Server                | Nombre JNDI                                    | Prefijo |
|-----------------------------|------------------------------------------------|---------|
| Domain Runtime MBean Server | weblogic.management.mbeanservers.domainruntime | /jndi/  |
| Runtime MBean Server        | weblogic.management.mbeanservers.runtime       | /jndi/  |
| Edit MBean Server           | weblogic.management.mbeanservers.edit          | /jndi/  |

El prefijo "/jndi/", se requiere para formar el nombre completo JNDI para realizar una conexión.

En la documentación este nombre JNDI lo conoceremos como *urlPath*.

Así, una las urlPath completas son:

```
/jndi/weblogic.management.mbeanservers.domainruntime
/jndi/weblogic.management.mbeanservers.runtime
/jndi/weblogic.management.mbeanservers.edit
¿Por qué hay 3 servidores de MBeans's dentro de un servidor WLS?
```

La respuesta primera es: para mantener cierta organización de los MBeans.

Debemos recordar que un dominio de Weblogic en la vida real está formado por varios servidores administrados, algunos de ellos quizás agrupados en *clusters*.

Aquí una descripción de cuál es el objetivo de cada uno de los servidores MBean.

| MBean Server                | Objetivo                                                                                                                                                                                                                                                                                            |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Domain Runtime MBean Server | Proporciona acceso a los MBeans de servicios del dominio completo, esto incluye aplicaciones desplegadas, servidores JMS, dataSources, etc.<br>Es el punto de acceso para acceder de manera jerárquica toda la información del dominio, es decir, de todos los servidores que conforman el dominio. |
| Runtime MBean Server        | Proporciona acceso a MBeans de tiempo de ejecución y puede activarlos dentro de la configuración.                                                                                                                                                                                                   |
| Edit MBean Server           | Proporciona el punto de entrada para administrar la Configuración del dominio.                                                                                                                                                                                                                      |

Ahondaremos un poco más en estos 3 servidores en próximas entregas, por ahora nos enfocaremos en el <Domain Runtime MBean Server>, vamos ahora a conocer un poco más sobre el código que usaremos.

#### El paquete javax.management

Como mencionábamos en la primera parte de esta serie, la implementación de JMX se encuentra en las clases principales de java prácticamente desde su inicio.

El paquete que contiene estas clases es: **javax.management.\***

Tal como se puede ver en: <https://docs.oracle.com/javase/8/docs/api/>

Las clases principales para operar con JMX se encuentran en estos paquetes:

| Paquete                     | Descripción                                                        |
|-----------------------------|--------------------------------------------------------------------|
| javax.management            | Proporciona las clases e interfaces para JMX en la plataforma.     |
| javax.management.loading    | Proporciona las clases que implementan la carga dinámica avanzada. |
| javax.management.modelmbean | Proporciona la definición de las clases ModelMBean.                |

|                             |                                                                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| javax.management.monitor    | Proporciona la definición para las clases de monitoreo.                                                                                                   |
| javax.management.openmbean  | Proporciona las clases necesarias para trabajar con Open MBeans.                                                                                          |
| javax.management.relation   | Proporciona la definición de un <i>Service Relation</i> .                                                                                                 |
| javax.management.remote     | Interfaces para realizar un acceso remoto a un servidor JMX.                                                                                              |
| javax.management.remote.rmi | El conector RMI es un conector para utilizar remotamente la API JMX, la cual usa RMI para transmitir peticiones desde el cliente hacia un servidor MBean. |
| javax.management.timer      | Proporciona la definición de un TimerMBean                                                                                                                |

Tabla 1. Paquetes javax.management y sus subpaquetes.

Conozcamos un poco más a detalle algunas de estas clases.

### MBeanServerConnection

La interface principal para operar sobre un servidor JMX se encuentra en el paquete:

javax.management.remote y es la interface: MBeanServerConnection

Esta es su descripción:

```
public interface MBeanServerConnection
```

Esta interfaz representa una forma de establecer comunicación con un servidor MBean, ya sea local o remoto.

Una de las interfaces que descienden directamente de esta interface es: MBeanServer la cual representa un servidor MBean local.

<https://docs.oracle.com/javase/8/docs/api/javax/management/MBeanServerConnection.html>

Si revisamos la documentación, veremos que los métodos que esta interface expone sirven para manipular casi todos los elementos que hemos revisado en el artículo anterior, podemos dar de alta un MBean, crear una notificación, recuperar los dominios (*¿recuerdas la parte del ObjectName?*), saber el número de MBeans de ese servidor, y muchos métodos más.

Pero, ¿Cómo obtener una instancia de alguna implementación de esta interface?

Veamos la siguiente imagen:

Figura 2. Relación de clases requeridas para generar un objeto MBeanServerConnection.

La imagen describe de manera gráfica como podemos generar una instancia de la implementación estándar de la interface MBeanServerConnection.

Lo primero que requerimos es un objeto JMXServiceUrl, éste sirve de parámetro de entrada para la clase JMXConnectorFactory.

La clase JMXConnectorFactory nos ayuda a generar una instancia de JMXConnector y esta a su vez nos genera lo que buscamos: una instancia de la implementación de MBeanServerConnection.

Veamos paso a paso como lograr esto.

### JMXServiceURL

Entonces, nuestro primer paso es generar un objeto de la clase: **JMXServiceURL**

```
public class javax.management.remote.JMXServiceURL
Representa la dirección de un servidor JMX (su punto de conexión).
Las instancias de esta clase son inmutables.
```

<https://docs.oracle.com/javase/8/docs/api/javax/management/remote/JMXServiceURL.html>

Esta clase tiene tres constructores, usaremos el más completo para iniciar nuestra conexión

```
JMXServiceURL(String protocol, String host, int port, String urlPath)
Constructs a JMXServiceURL with the given parts.
```

Como <protocol> y para este ejemplo utilizaremos t3, <port> es el puerto es que nos da WLS por default, el 7001, (verifica si por este puerto puedes acceder a la consola web), el usuario y el password debe estar en nuestro poder y solo hace falta la última parte: el urlPath

El *path* lo tomamos de la tabla que nos indica los 3 servidores disponibles en un dominio weblogic server.

Tenemos todos los datos necesarios para crear una instancia:

```
String hostname = "localhost";
String portString = "7001";
String username = "weblogic";
String password = "manager1";
String protocol = "t3";
Integer portInteger = Integer.valueOf(portString);
int port = portInteger.intValue();
String jndiroot = "/jndi/";
String mserver = "weblogic.management.mbeanservers.domainruntime";
String urlPath = jndiroot + mserver;
String jndiroot = "/jndi/";
String mserver = "weblogic.management.mbeanservers.domainruntime";
JMXServiceURL serviceURL = new JMXServiceURL(protocol, hostname,
    port, jndiroot + mserver);
```

### JMXConnectorFactory

Constructor

public class **JMXConnectorFactory**  
Fábrica para crear conectorores-cliente JMX. No hay instancias de esta clase.  
  
No hay constructor para esta clase, es una Fábrica.

Esta clase no tiene constructores ya que es una *fábrica*, es decir, su labor es generar objetos a partir de ciertos parámetros de entrada.  
La implementación del *patrón fabrica* por lo general deriva en métodos estáticos, así, en este caso el método que vamos a utilizar es:

static JMXConnector connect(JMXServiceURL serviceURL, Map<String,?> environment)  
Crea una conexión hacia un servidor dada la información proporcionada.

**Crear una conexión.**  
Tenemos ya nuestro serviceURL, solo nos falta un <Map> con algunas propiedades requeridas para concretar la conexión.  
Para este ejemplo, crearemos un mapa con los siguientes valores:

| Liave                                          | Descripción              |
|------------------------------------------------|--------------------------|
| Context.SECURITY_PRINCIPAL                     | Usuario.                 |
| Context.SECURITY_CREDENTIALS                   | Password.                |
| JMXConnectorFactory.PROTOCOL_PROVIDER_PACKAGES | Paquete de conexión.     |
| jmx.remote.x.request.waiting.timeout           | Timeout en milisegundos. |

Así queda nuestro código:

```
private JMXConnector connector;  
//...  
Hashtable environment = new Hashtable();  
environment.put(Context.SECURITY_PRINCIPAL, username);  
environment.put(Context.SECURITY_CREDENTIALS, password);  
environment.put(JMXConnectorFactory.PROTOCOL_PROVIDER_PACKAGES,  
"weblogic.management.remote");  
environment.put("jmx.remote.x.request.waiting.timeout", new Long(10000));  
connector = JMXConnectorFactory.connect(serviceURL, environment);
```

En <connector> tenemos ya una instancia de <JMXConnector>, conozcamos un poco más a detalle esta interface.

**JMXConnector**  
Constructor

public interface **JMXConnector**  
El cliente final de un conector JMX. Un objeto de este tipo se puede utilizar para establecer una conexión a un servidor (Conector).

Los constructores solo pueden existir en las implementaciones de las interfaces.

La interface tiene el siguiente método:

**getMBeanServerConnection**  
La descripción del método dice:

*Devuelve un objeto MBeanServerConnection que representa un servidor MBean remoto.*  
Es con este método que logramos obtener nuestro objeto <MBeanServerConnection>.

```
private MBeanServerConnection connection;  
connection = connector.getMBeanServerConnection();
```

Tenemos por fin lo que buscamos, ahora, ya que tenemos la instancia, demos un repaso a los métodos a los cuales tenemos acceso.

**MBeanServerConnection y sus métodos.**  
El siguiente es el listado de métodos públicos de MBeanServerConnection:

| Método.                                                                                                                   | Descripción.                                            |
|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| void addNotificationListener(ObjectName name, NotificationListener listener, NotificationFilter filter, Object handback)  | Agrega un <listener> a un MBean ya registrado.          |
| void addNotificationListener(ObjectName name, ObjectName listener, NotificationFilter filter, Object handback)            | Agrega un <listener> a un MBean ya registrado.          |
| ObjectInstance createMBean(String className, ObjectName name)                                                             | Instancia y registra un MBean en el Servidor de MBeans. |
| ObjectInstance createMBean(String className, ObjectName name, Object[] params, String[] signature)                        | Instancia y registra un MBean en el Servidor de MBeans. |
| ObjectInstance createMBean(String className, ObjectName name, ObjectName loaderName)                                      | Instancia y registra un MBean en el Servidor de MBeans. |
| ObjectInstance createMBean(String className, ObjectName name, ObjectName loaderName, Object[] params, String[] signature) | Instancia y registra un MBean en el Servidor de MBeans. |
|                                                                                                                           |                                                         |

|                                                                                                                             |                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Object getAttribute(ObjectName name, String attribute)                                                                      | Obtiene el valor de un atributo específico de un MBean dado su <ObjectName>.                               |
| AttributeList getAttributes(ObjectName name, String[] attributes)                                                           | Recupera los valores de varios atributos de un MBean, dado su <ObjectName>.                                |
| String getDefaultDomain()                                                                                                   | Devuelve el dominio predeterminado utilizado para nombrar MBeans en el servidor.                           |
| String[] getDomains()                                                                                                       | Devuelve la lista de dominios con los que está registrado cualquier MBean del servidor.                    |
| Integer getMBeanCount()                                                                                                     | Devuelve el número de MBeans registrados en el servidor MBean.                                             |
| MBeanInfo getMBeanInfo(ObjectName name)                                                                                     | Este método descubre los atributos y las operaciones que un MBean expone para su gestión.                  |
| ObjectInstance getObjectInstance(ObjectName name)                                                                           | Obtiene la <i>ObjectInstance</i> de un MBean dado, registrado con el servidor MBean.                       |
| Object invoke(ObjectName name, String operationName, Object[] params, String[] signature)                                   | Invoca una operación en un MBean.                                                                          |
| boolean isInstanceOf(ObjectName name, String className)                                                                     | Devuelve true si el MBean especificado es una instancia de la clase especificada, false en caso contrario. |
| boolean isRegistered(ObjectName name)                                                                                       | Comprueba si un MBean, identificado por su nombre de objeto, ya está registrado en el servidor MBean.      |
| Set<ObjectInstance> queryMBeans(ObjectName name, QueryExp query)                                                            | Obtiene los MBeans controlados por el servidor MBean.                                                      |
| Set<ObjectName> queryNames(ObjectName name, QueryExp query)                                                                 | Obtiene los nombres de MBeans controlados por el servidor MBean.                                           |
| Void removeNotificationListener(ObjectName name, NotificationListener listener)                                             | Elimina un <listener> de un MBean registrado.                                                              |
| Void removeNotificationListener(ObjectName name, NotificationListener listener, NotificationFilter filter, Object handback) | Elimina un <listener> de un MBean registrado.                                                              |
| Void removeNotificationListener(ObjectName name, ObjectName listener)                                                       | Elimina un <listener> de un MBean registrado.                                                              |
| void removeNotificationListener(ObjectName name, ObjectName listener, NotificationFilter filter, Object handback)           | Elimina un <listener> de un MBean registrado.                                                              |
| Void setAttribute(ObjectName name, Attribute attribute)                                                                     | Establece el valor de un atributo específico de un MBean dado su <ObjectName>.                             |
| AttributeList setAttributes(ObjectName name, AttributeList attributes)                                                      | Establece los valores de varios atributos de un MBean dado su <ObjectName>.                                |
| Void unregisterMBean(ObjectName name)                                                                                       | Anula el registro de un MBean del servidor MBean.                                                          |

Tabla 2. Métodos de la clase *MBeanServerConnection*.

¿Te suena familiar la descripción de estos métodos?, debe de serlo pues cubren en su totalidad lo que hemos comentado en el *nivel de instrumentación* de la especificación.

De todos los métodos, por ahora usaremos <queryNames> para recuperar el listado de MBeans del servidor al cual nos conectaremos.

#### queryNames

Set<ObjectName> queryNames(ObjectName name, QueryExp query) throws IOException  
Obtiene los nombres de MBeans controlados por el MBeanServer.

Se puede usar de la siguiente manera: Obtiene un listado de nombre que coincidan por *patter-matching* con el primer parámetro <name> y/o que coincidan con la expresión indicada en el parámetro <query>.

Cuando el parámetro <name> es null o no se especifican un dominio, se recuperan todos los MBeans. Al final debe de retornar un <Set> de <ObjectName's>

Entonces, dada la documentación, si colocamos null en ambos parámetros de entrada vamos a recuperar todos los nombres de los MBeans que se encuentren en ese servidor.

El código sería algo como esto:

```
Set<ObjectName> mbeans = connection.queryNames(null, null);
```

Ya tenemos todas las piezas para recuperar el listado de MBeans del servidor de MBeans.

#### Listado de MBeans.

Nuestro primer ejemplo de código tiene como objetivo conectarse a uno de estos 3 servidores, una vez realizada la conexión y haciendo uso de la API estándar recuperaremos el listado de todos los MBeans expuestos en dicho servidor.

Este código está basado en el ejercicio que aparece en: <http://middlewaremagic.com/weblogic/?p=185>

### Listado del código. Ejercicio 1.

```
import java.io.IOException;
import java.net.MalformedURLException;
import java.util.Hashtable;
import java.util.Set;
import javax.management.MBeanServerConnection;
import javax.management.ObjectName;
import javax.management.remote.JMXConnector;
import javax.management.remote.JMXConnectorFactory;
import javax.management.remote.JMXServiceURL;
import javax.naming.Context;

/**
 * Este ejemplo esta basado en el mostrado en:
 * http://middlewaremagic.com/weblogic/?p=185 de Jay SenSharma
 *
 * @author RuGI (S&P Solutions S.A de C.V)
 */
public class WLS_ALL_JMX_URLS {

    private MBeanServerConnection connection;
    private JMXConnector connector;

    public void initConnection(String hostname, String portString,
        String username, String password) throws IOException,
        MalformedURLException {
        String protocol = "t3";
        Integer portInteger = Integer.valueOf(portString);
        int port = portInteger.intValue();
        String jndiroot = "/jndi/";
        String mserver = "weblogic.management.mbeanservers.domainruntime";
        String urlPath = jndiroot + mserver;
        JMXServiceURL serviceURL = new JMXServiceURL(protocol, hostname,
            port, urlPath );
        Hashtable environment = new Hashtable();
        environment.put(Context.SECURITY_PRINCIPAL, username);
        environment.put(Context.SECURITY_CREDENTIALS, password);
        environment.put(JMXConnectorFactory.PROTOCOL_PROVIDER_PACKAGES,
            "weblogic.management.remote");
        environment.put("jmx.remote.x.request.waiting.timeout", new Long(10000));
        connector = JMXConnectorFactory.connect(serviceURL, environment);
        connection = connector.getMBeanServerConnection();
    }

    public void showAllMBeans() throws Exception {
        if (connection != null) {
            Set<ObjectName> mbeans = connection.queryNames(null, null);
            for (ObjectName mbeanName : mbeans) {
                System.out.println(mbeanName);
            }
        } else {
            System.out.println("Aun no hay conexion.");
        }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("-----");
        String hostname = "localhost";
        String portString = "7001";
        String username = "weblogic";
        String password = "welcome1";
        WLS_ALL_JMX_URLS all = new WLS_ALL_JMX_URLS();
        all.initConnection(hostname, portString, username, password);
        all.showAllMBeans();
    }
}
}
}
```

### Validación de credenciales

Antes de ejecutar el código, debemos validar que el servidor se encuentra funcionando adecuadamente y que las credenciales son válidas. Es importante validar todos los datos de la conexión, comenzando por la URL:

<http://localhost:7001/console>

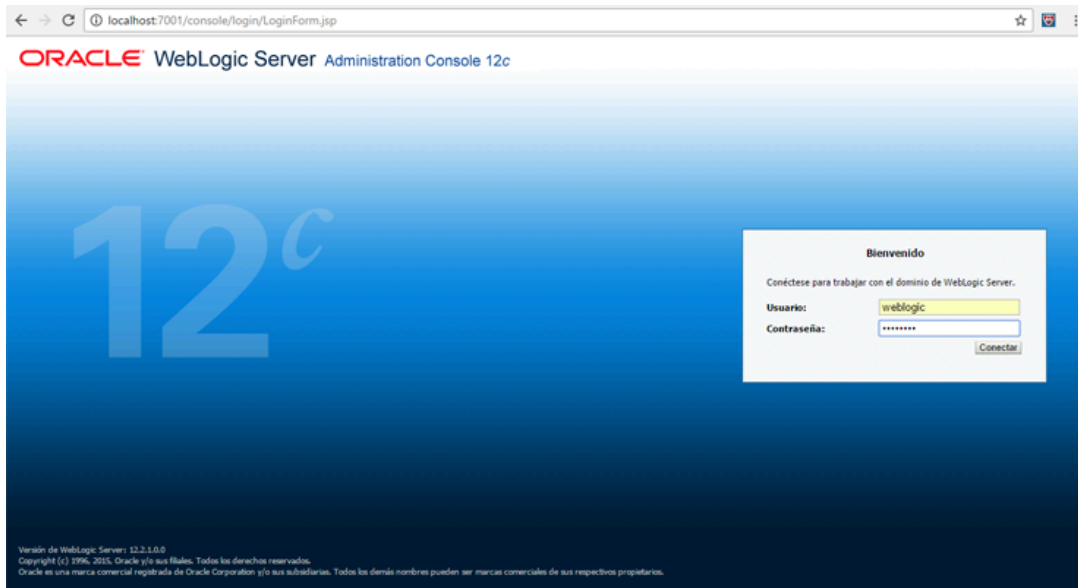


Figura 3. Formulario de acceso a WLS 12.2.1.

Una vez validada la url, debemos utilizar las credenciales y ver que el acceso es correcto.

|          |          |
|----------|----------|
| Usuario  | weblogic |
| Password | welcome1 |

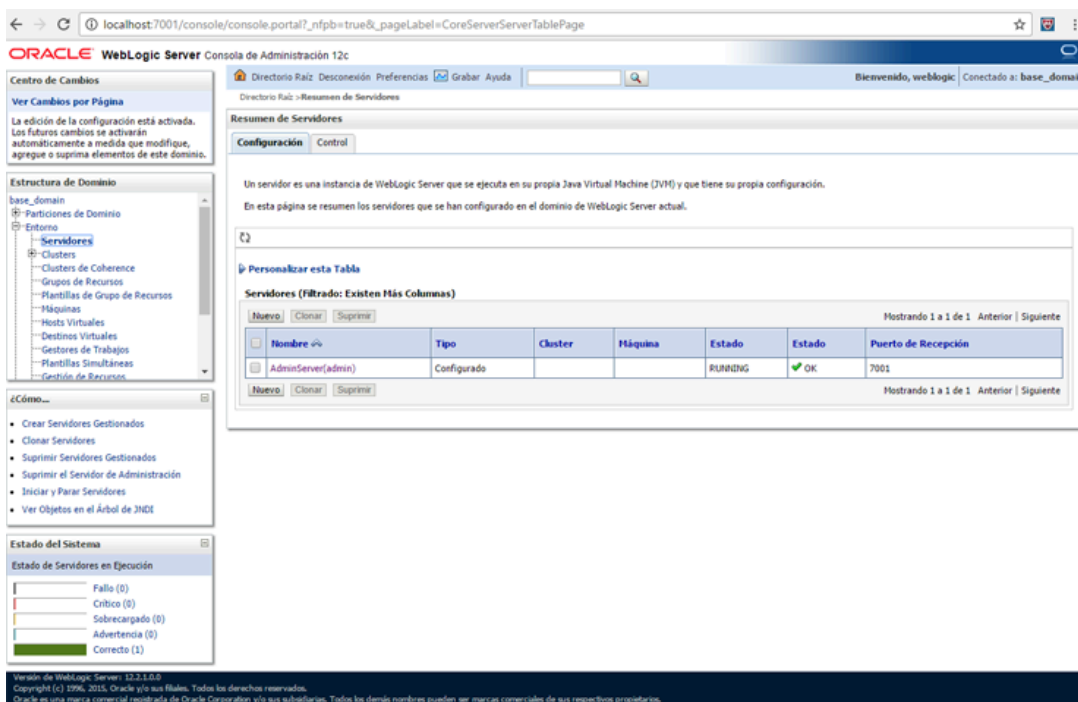


Figura 4. Consola web de WLS 12.2.1.

Listo, con esto ya podemos probar el código.

### Ejecución

Una vez que validemos las credenciales y éstas sean las mismas que usamos en el código, podemos compilarlo.

```
Mode      LastWriteTime          Length Name
-----
-a----    28/12/2016 09:17 p.m.    2602WLS_ALL_JMX_URLS.java
```

```
%> javac .\WLS_ALL_JMX_URLS.java
```

Y ejecutarlo:

```
%> java WLS_ALL_JMX_URLS
```

Al ejecutarlo es casi seguro que obtengamos esta excepción.

```
Exception in thread "main" java.net.MalformedURLException: Unsupported protocol: t3
at javax.management.remote.JMXConnectorFactory.newJMXConnector(JMXConnectorFactory.java:359)
at javax.management.remote.JMXConnectorFactory.connect(JMXConnectorFactory.java:269)
```

T3 es una implementación propietaria de RMI (también existe T3S) y la excepción que hemos obtenido nos indica que dicho protocolo no está soportado (*no encuentra clases para soportarlo*), lo único que debemos hacer es agregar un par de jars a nuestro classpath.

**Wljmxclient.jar y wlclient.jar**

Ambos archivos se encuentran dentro de la instalación de nuestro dominio de WLS 12.2.1

```
ORACLE_HOME\wlserver\server\lib\wljmxclient.jar
ORACLE_HOME\wlserver\server\lib\wlclient.jar
```

Sólo debemos agregar el jar, al classpath, una manera es copiando los archivos en la misma carpeta donde se encuentra nuestra clase:

```
-a---      31/08/2016   10:39 p. m.    2655080 wlclient.jar
-a---      31/08/2016   10:39 p. m.    241028  wljmxclient.jar
-a-----   28/12/2016    09:17 p. m.    2602   WLS_ALL_JMX_URLS.java
```

No hace falta volver a compilar ya que, las clases que nos proporcionan estos 2 jars se utilizan en tiempo de ejecución.

Para ejecutar, sólo debemos agregar esos dos jars y la carpeta actual al classpath. Para indicar la carpeta actual se usa el punto ".", el separador del classpath en este ejemplo es el punto y coma para windows y dos puntos para \*nix.

Entonces para ejecutar en Windows sólo haremos lo siguiente:

```
%> java -cp "wlclient.jar;wljmxclient.jar;%;" WLS_ALL_JMX_URLS
```

Si después de agregar el jar ahora te aparece una excepción como la siguiente:

```
Caused by: javax.naming.NamingException: Unhandled exception in lookup
[Root exception is org.omg.CORBA.NO_PERMISSION: vmcid: 0x0 minor code: 0 completed: No]
at weblogic.corba.j2ee.naming.Utils.wrapNamingException(Utils.java:83)
at weblogic.corba.j2ee.naming.ContextImpl.lookup(ContextImpl.java:253)
at weblogic.corba.j2ee.naming.ContextImpl.lookup(ContextImpl.java:191)
at javax.naming.InitialContext.lookup(InitialContext.java:417)
at weblogic.management.remote.common.ClientProviderBase.makeConnection(ClientProviderBase.java:280)
... 5 more
Caused by: org.omg.CORBA.NO_PERMISSION: vmcid: 0x0 minor code: 0 completed: No
at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
at
```

Es probable que estés proporcionado mal alguno de los datos para firmarte, valida nuevamente usuario y password.

Una vez agregado ambos jar's al classpath y colocando las credenciales adecuadas, debemos obtener una respuesta similar a la siguiente:

```
java.util.logging:Location=AdminServer,type=Logging
com.bea:Name=base_domain,Location=base_domain,Type=GzipCompression,WebAppContainer=base_domain
com.bea:Name=oracle.sdp.client#2.0@12.2.1,Location=base_domain,Type=Library
com.bea:Name=Default[http][2],ServerRuntime=AdminServer,Location=AdminServer,Type=ServerChannelRuntime
com.bea:Name=AdminServer,Location=base_domain,Type=OverloadProtection,Server=AdminServer
com.oracle:type=OVD,context=ids,name=AdaptersConfig
com.bea:Name=base_domain,Location=AdminServer,Type=CdiContainer
```

Puedes ver el listado completo de MBeans en el siguiente gits:  
<https://gist.github.com/rugi/41ccf03f9f2a6eb47d60bd6eb05830dc>

Son más de 1000 MBeans y su respetivo ObjectName, lo primero que llama la atención es que los ObjectName son muy largos.

Esto tiene una razón de ser.

**Weblogic y sus ObjectName's**

Después de dar un vistazo rápido por los ObjectNames de la lista seguramente te has percatado que, el dominio (todo aquello antes de los dos puntos), es casi siempre el mismo, además, existen propiedades que se vuelven comunes: Name, Location y Type.

Por definición los ObjectNames no proporcionan un mecanismo para organizar o jerarquizar los MBeans, por ello, WLS intenta cubrir estas necesidades con las propiedades en el ObjectName.

El dominio por default para un ObjectName en WLS es:

```
com.bea
```

Las principales propiedades de un objectName son:

|          |                                                                                                                                                                                                                                       |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Name     | Es el nombre principal del ObjectName.                                                                                                                                                                                                |
| Location | Indica la ubicación del ObjectName.<br><i>Nota: Debemos recordar que estamos accediendo al AdminServer y que, existen escenarios donde existen más servidores (servidores administrados).</i>                                         |
| Type     | Indica el tipo de MBean, por lo general es un valor contextual, es decir:<br>Si el MBean representa un componente de software, este campo nos dirá qué tipo de componente es.<br>Si es un punto de configuración también lo indicará. |

Realmente hay un par de propiedades más que son importantes de conocer, pero, por ahora con está 3 podemos continuar con nuestro objetivo.

**Recuperar un valor.**

Ahora que ya sabemos cómo se forman los objectNames en Weblogic, vamos a recuperar un MBean en especial. Con este ObjectName accedemos a un MBean que nos proporciona toda la información del AdminServer.

Este MBean tiene el siguiente ObjectName (Si lo buscas en el listado de MBeans anterior, debes de localizarlo):

```
com.bea:Name=DomainRuntimeService,Type=weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean
```

Algo interesante de este MBean es que, su propiedad Type nos indica el nombre del MBean que lo implementa:



Type weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean

Con esto, podemos ir a la documentación y ver que nos dice:

Proporciona un punto de acceso común para navegar a todos los MBeans: los de configuración del dominio y los que se existen en tiempo de ejecución, así como a los MBeans que proporcionan servicios de todo el dominio.

<https://docs.oracle.com/middleware/1221/wls/WLAPI/weblogic/management/mbeanservers/domainruntime/DomainRuntimeServiceMBean.html>

Lo primero que requerimos es acceder a ese MBean, y como ahora lo sabemos, esto se logra a través de su ObjectName:

```
private static final ObjectName service;
...

try {
    service = new ObjectName("com.bea:Name=DomainRuntimeService,
        Type=weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean");
} catch (MalformedObjectNameException e) {
    throw new AssertionError(e.getMessage());
}
```

DomainRunTimeServiceMBean está compuesto, como casi todas las clases en java, por una serie de elementos internos.

La siguiente imagen muestra algunas propiedades que nos interesan para concluir nuestro ejercicio.

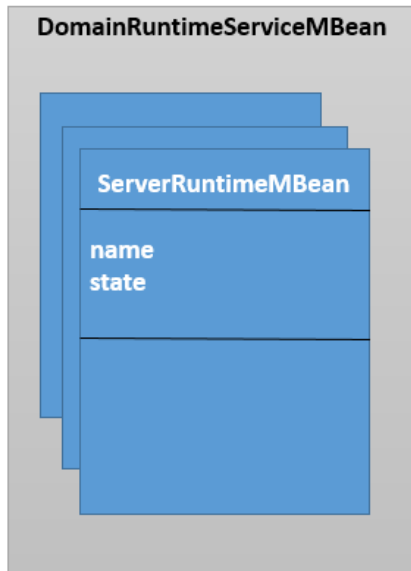


Figura 5. Composición parcial de la clase DomainRuntimeServiceMBean.

La composición de clases permite que, una clase esté compuesta por otras clases.

De la clase DomainRunTimeServiceMBean, nos interesa su propiedad: ServerRuntimes.

|                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|
| <b>ServerRuntimes</b><br>Contiene un arreglo de MBeans ServerRuntimeMBean, cada uno representa un servidor de nuestro dominio. |
|--------------------------------------------------------------------------------------------------------------------------------|

ServerRuntimeMBean es, como su nombre lo indica, otro MBean. Este MBean tiene dos propiedades que nos servirán para terminar nuestro ejemplo:

|                                                                     |
|---------------------------------------------------------------------|
| <b>Name</b><br>El nombre de esta configuración.                     |
| <b>State</b><br>El estado del server según el ciclo de vida de WLS. |

¿Cómo recuperaremos estos valores? El método como seguramente lo estás imaginando es: getAttribute.

### getAttribute

Este método solo recibe un ObjectName y la propiedad (en string) que queremos recuperar de ese ObjectName.

Devuelve el objeto cuyo nombre responde al de la propiedad indicada (por lo general este objeto es a su vez otro ObjectName), por lo que, la forma para recuperar valores es similar en todos los casos.

Continuemos con el ejercicio.

Recuperemos primero los ServerRuntimes

```
public static ObjectName[] getServerRuntimes() throws Exception {
    return (ObjectName[]) connection.getAttribute(service, "ServerRuntimes");
}
```

Ya que tenemos el objeto ServerRuntimes (realmente es un arreglo de objetos<ObjectName>, pero un arreglo *per se*, es un objeto), vamos ahora por el nombre y estado de cada uno.

(Observa como ahora getAttribute se invoca ahora con cada uno de los elementos del arreglo y no sobre service, como en el primer caso, estamos cambiando el primer parámetro de entrada ya que, las propiedades que queremos recuperar se encuentran en ObjectNames distintos).

```
public void printNameAndState() throws Exception {
    ObjectName[] serverRT = getServerRuntimes();
}
```

```

System.out.println("got server runtimes");
int length = (int) serverRT.length;
for (int i = 0; i < length; i++) {
String name = (String) connection.getAttribute(serverRT[i], "Name");
String state = (String) connection.getAttribute(serverRT[i], "State");
System.out.println("Server name: " + name + ". Server state: " + state);
}
}
}

```

Así queda nuestro código completo:

### Listado completo. Ejercicio 2:

```

/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
import java.io.IOException;
import java.net.MalformedURLException;
import java.util.Hashtable;
import javax.management.MBeanServerConnection;
import javax.management.MalformedObjectNameException;
import javax.management.ObjectName;
import javax.management.remote.JMXConnector;
import javax.management.remote.JMXConnectorFactory;
import javax.management.remote.JMXServiceURL;
import javax.naming.Context;

/**
 * Tomado de https://docs.oracle.com/cd/E13222_01/wls/docs90/jmx/editWLS.html
 *
 * @author Oracle Co.
 */
public class PrintServerState {

    private static MBeanServerConnection connection;
    private static JMXConnector connector;
    private static final ObjectName service;

    // Initializing the object name for DomainRuntimeServiceMBean
    // so it can be used throughout the class.
    static {
        try {
            service = new ObjectName("com.bea:Name=DomainRuntimeService,
Type=weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean");
        } catch (MalformedObjectNameException e) {
            throw new AssertionError(e.getMessage());
        }
    }

    /**
     * Initialize connection to the Domain Runtime MBean Server
     */
    public static void initConnection(String hostname, String portString,
String username, String password) throws IOException,
MalformedURLException {

        String protocol = "t3";
        Integer portInteger = Integer.valueOf(portString);
        int port = portInteger.intValue();
        String jndiroot = "/jndi/";
        String msrver = "weblogic.management.mbeanservers.domainruntime";
        JMXServiceURL serviceURL = new JMXServiceURL(protocol, hostname,
port, jndiroot + msrver);
        Hashtable environment = new Hashtable();
        environment.put(Context.SECURITY_PRINCIPAL, username);
        environment.put(Context.SECURITY_CREDENTIALS, password);
        environment.put(JMXConnectorFactory.PROTOCOL_PROVIDER_PACKAGES,
"weblogic.management.remote");
        environment.put("jmx.remote.x.request.waiting.timeout", new Long(10000));
        connector = JMXConnectorFactory.connect(serviceURL, environment);
        connection = connector.getMBeanServerConnection();
    }

    /**
     * Print an array of ServerRuntimeMBeans.
     * This MBean is the root of the runtime MBean hierarchy, and
     * each server in the domain hosts its own instance.
     */
    public static ObjectName[] getServerRuntimes() throws Exception {
        return (ObjectName[]) connection.getAttribute(service, "ServerRuntimes");
    }

    /**
     * Iterate through ServerRuntimeMBeans and get the name and state
     */
    public void printNameAndState() throws Exception {
        ObjectName[] serverRT = getServerRuntimes();
        System.out.println("got server runtimes");
        int length = (int) serverRT.length;
        for (int i = 0; i < length; i++) {
            String name = (String) connection.getAttribute(serverRT[i], "Name");
            String state = (String) connection.getAttribute(serverRT[i], "State");
            System.out.println("Server name: " + name + ". Server state: " + state);
        }
    }
}

```

```
    } // for
  } // method

  public static void main(String[] args) throws Exception {
    String hostname = "localhost";
    String portString = "7001";
    String username = "weblogic";
    String password = "welcome1";
    PrintServerState s = new PrintServerState();
    initConnection(hostname, portString, username, password);
    s.printNameAndState();
    connector.close();
  }
}
```

Estamos listos para compilar y ejecutar.

Primero compilamos:

```
%> javac .\PrintServerState.java
```

Y ejecutamos:

```
%> java -cp "wljmxclient.jar;wlcient.jar;." PrintServerState
```

Debemos ver una salida similar a la siguiente:

```
got server runtimes
Server name: AdminServer.   Server state: RUNNING
```

Listo, hemos recuperado propiedades de un MBean dentro de un WLS 12.2.1

Puedes conseguir el código utilizado en el repositorio indicado en los enlaces que vienen al final de este documento.

**Conclusión.**

En esta segunda entrega hemos conocido la manera de conectarnos remotamente a cualquiera de los 3 servidores de MBeans que expone WLS, sabemos ya el porqué de la nomenclatura de cada ObjectName y podemos ubicar a detalle la documentación de cada uno.

Utilizando código estándar hemos accedido a un servidor de MBeans, y recuperado un valor de alguno de los distintos MBeans. Seguramente te estás preguntando ¿Por qué no probamos ya la invocación de métodos? Y la respuesta es: hay cierta teoría que debemos comprender antes.

En siguientes entregas conoceremos sobre esta teoría, más sobre la distribución de los MBeans en Weblogic e inspeccionaremos un dominio no tan sencillo.

**Enlaces.**

- JSR-000003: JavaTM Management Extensions (JMX) Specification.  
<https://jcp.org/en/jsr/detail?id=3>
- JSR-000003 JavaTM Management Extensions (JMX) (Maintenance Release 4)  
<https://jcp.org/aboutJava/communityprocess/mrel/jsr003/index4.html>
- JSR 000255 - JMX Specification, version 2.0  
<https://jcp.org/en/jsr/detail?id=255>
- JSR 160: Java Management Extensions (JMX) Remote API.  
<https://jcp.org/en/jsr/detail?id=160>
- Oracle® Fusion Middleware Developing Custom Management Utilities Using JMX for Oracle WebLogic Server.  
<https://docs.oracle.com/middleware/1221/wls/JMXCU/title.htm>
- Oracle Fusion Middleware Java API Reference for Oracle WebLogic Server 12c (12.2.1)  
<https://docs.oracle.com/middleware/1221/wls/WLAPI/index.html>
- Repositorio con el código usado en esta serie de artículos.  
<https://github.com/rugi/jmx>

**Isaac Ruiz Guerra** (@rugi), es programador Java y Consultor TI. Especializado en integración de sistemas, fundamentalmente relacionados con el sector financiero. Actualmente forma parte del equipo de S&P Solutions. Escribe en su blog personal xhubacubi.blogspot.com, pero también participa y pertenece a [www.javahispano.org](http://www.javahispano.org) y a [www.javamexico.org](http://www.javamexico.org)

*Este artículo ha sido revisado por el equipo de productos Oracle y se encuentra en cumplimiento de las normas y prácticas para el uso de los productos Oracle.*

 E-mail this page  Printer View  Printer View

**Contáctenos**

- Ventas en EE. UU.:  
+1.800.633.0738
- Reciba una llamada de Oracle
- Contactos globales
- Directorio de soporte

**Acerca de Oracle**

- Información de la empresa
- Comunidades

**Cloud**

- Descripción general de Cloud Solutions
- Software (SaaS)
- Plataforma (PaaS)
- Infraestructura (IaaS)
- Datos (DaaS)
- Prueba gratuita de la nube

**Eventos**

**Acciones principales**

- Descargue Java
- Descargue Java para desarrolladores
- Pruebe Oracle Cloud
- Suscríbase a los correos electrónicos

**Noticias**

- Sala de prensa
- Revistas

**Temas clave**

- ERP, EPM (Finanzas)
- HCM (RR. HH. y talento)
- Mercadeo
- CX (Ventas, servicio, comercio)
- Soluciones sectoriales
- Base de datos
- MySQL
- Middleware
- Java

[Carreras](#)

[Oracle Code](#)

[Historias de éxito de los clientes](#)

[Sistemas de ingeniería](#)

[JavaOne](#)

[Blogs](#)

[Todos los eventos de Oracle](#)